

Práctica 5

Transmisión OFDM

Índice

1. Generación y recepción de símbolos OFDM
2. Estimación y compensación de canal
3. Efectos errores de sincronización
4. Métodos de sincronización
5. Receptor OFDM sincronizado

1. Generación y recepción OFDM

- Emplearemos las funciones:

```
sofdm = ofdmmod(Xda,Nfft,Ncp,I_nu,I_pi,Xpi)
```

```
[Yda,Ypi] = ofdmdemod(sofdm,Nfft,Ncp,retIni,I_nu,I_pi)
```

- Xda: vector datos símbolos QAM; Nfft: tamaño FFT; Ncp: long CP; I_nu: índices portadoras nulas; I_pi: índices portadoras piloto; Xpi: vector símbolos piloto (PSK)
- Yda: vector símbolos QAM recibido; Ypi: vector símbolos piloto recibidos; **retIni: entre 0 y Ncp (por defecto Ncp)**

- Para la modulación QAM:

```
Xda = qammod(Bits_T,Mqam,InputType="bit",UnitAveragePower=true)
```

```
Bits_R = qamdemod(Xda,Mqam,OutputType="bit",UnitAveragePower=true)
```

- Cálculo de los índices de las portadoras nulas:
 - I_{DC} es la posición de continua
 - La AF (en la FFT es -0.5 o $F_s/2$) está en la posición 1
 - N_{da} : número de subportadoras datos (par, mitad alrededor DC)
 - N_{pi} : número de subportadoras piloto (par)
 - N_{nu} : número de subportadoras nulas
 - N_{ba} : número de subportadoras de guarda

`%% Ejemplo WLAN`

`Nfft = 64;`

`Nda = 48;`

`Npi = 4;`

`I_DC = Nfft/2+1`

`Nnu = Nfft - Nda - Npi`

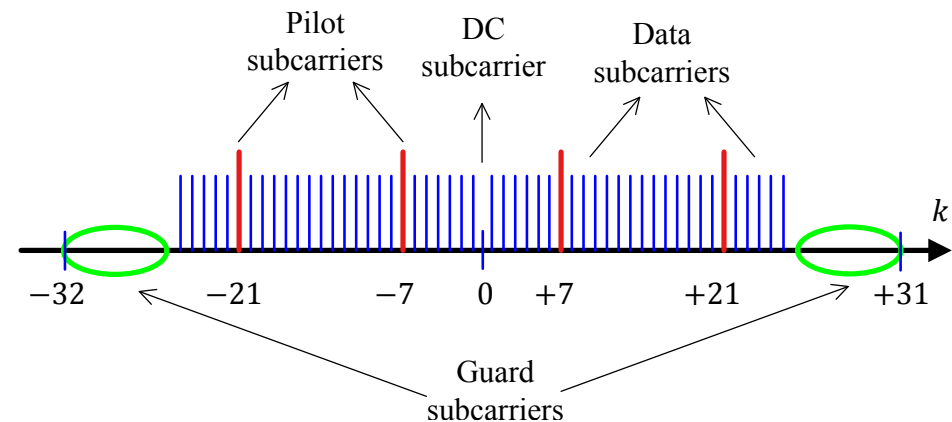
`Nba = (Nnu - 2)/2`

`I_nu = [(1:Nba+1) I_DC (Nfft-Nba+1:Nfft)]'`

`length(I_nu)`

`I_pi = [-21 -7 7 21]' + (Nfft/2+1);`

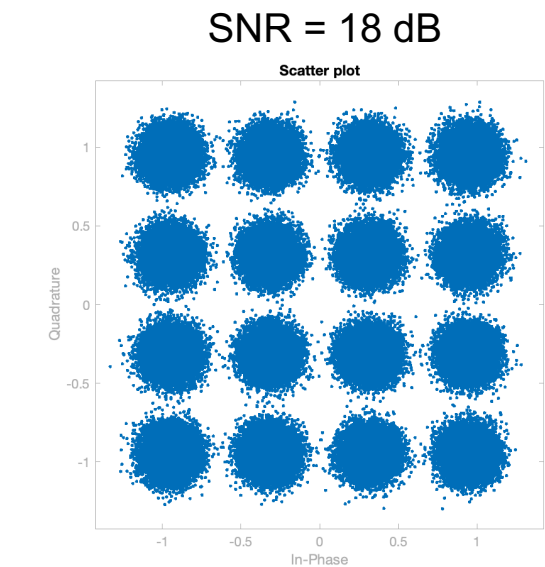
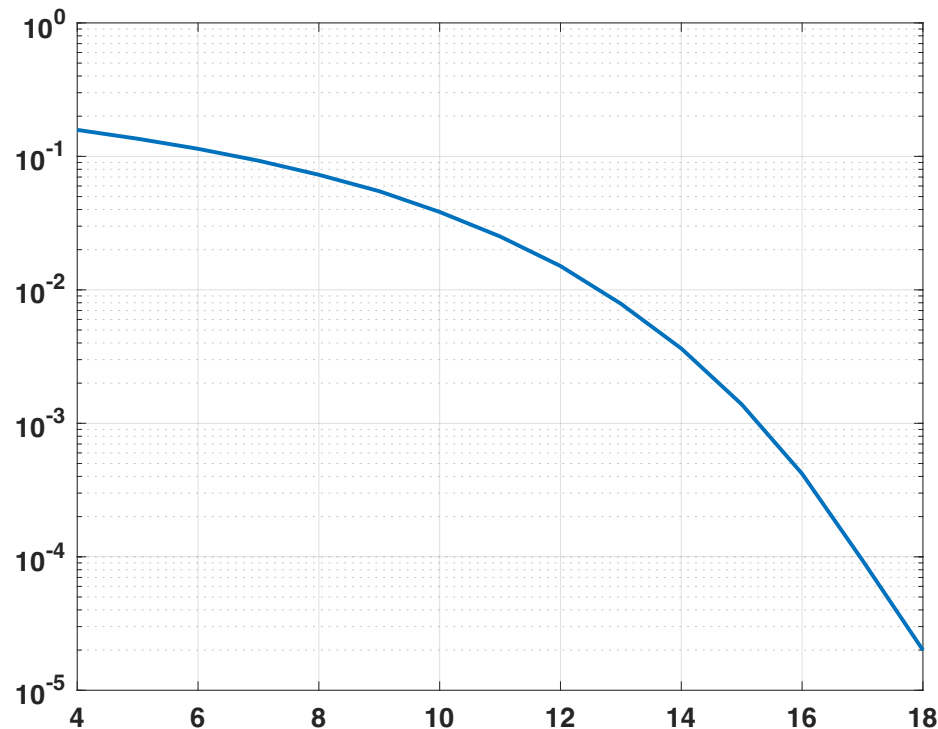
`I_da = setdiff(1:Nfft,[I_nu(:); I_pi(:)]);`



- Ejemplo (*antes inicializa variables WLAN previas*):

```
Mqam = 16;  
NSofdm = 10;    %Número de símbolos OFDM;  
Ncp = 16;  
  
Xpi = zeros(Npi,NSofdm);  
  
Bits_T = randi([0 1],Nda*log2(Mqam),NSofdm);  
Xda = qammod(Bits_T,Mqam,InputType="bit",UnitAveragePower=true);  
  
sofdm = ofdmmod(Xda,Nfft,Ncp,I_nu,I_pi,Xpi);  
  
[Yda,Ypi] = ofdmdemod(sofdm,Nfft,Ncp,Ncp,I_nu,I_pi);  
  
Bits_R = qamdemod(Yda,Mqam,OutputType="bit",UnitAveragePower=true);  
  
[n_er,ber] = biterr(Bits_T,Bits_R)
```

- Gráfica de prestaciones en AWGN:
 - En un *script* nuevo:
 - Crea una sección de parámetros con los valores de OFDM para WLAN
 - Calcula el NSofdm necesario para transmitir 1e6 bits con 16QAM
 - Calcula los bits y genera los símbolos OFDM
 - Define SNRdB = 4:18 y un vector BER de ceros del mismo tamaño
 - Crea un bucle for que recorra los índices de SNRdB y añada ruido a los símbolos, los demodule y calcule el BER(k)



Estructura de la simulación:

1. Define todos los parámetros para generar OFDM como en WLAN
2. Calcula el `NSofdm` necesario para transmitir 1e6 bits con 16QAM (4 bits):
 - Cada símbolo OFDM transporta = 4 bits · Num. portadoras datos
 - `NSofdm` debe ser un número entero superior (`ceil`)
3. Genera los bits y úsalos para crear los símbolos OFDM
4. Define `SNRdB = 4:18` y un vector `BER` de ceros del mismo tamaño (puedes usar `length` para definir el tamaño del vector de ceros)
5. Crea un bucle (`for k=1:L`) para recorrer los índices de `SNRdB`:
 - A. Añade ruido a la señal OFDM con `awgn`
 - B. Demodula la señal OFDM a la salida del canal AWGN y obtén los bits recibidos
 - C. Calcula el `BER(k)` con `biterr`
6. Dibuja la gráfica de BER vs. SNR con `semilogy`

Instrucciones de Matlab:

- `sofdm_ruido = awgn(sofdm,SNRdB(k),'measured');`
- `semilogy(SNRdB,BER)`
- `scatterplot(Yda(:))`

2. Estimación y compensación canal

- En OFDM se suele transmitir una señal de referencia para estimar el canal:
 - Tiene los mismos parámetros que la señal OFDM
 - Las portadoras de datos y piloto se cargan con símbolos BPSK
 - El receptor conoce qué símbolos BPSK se han usado
- Vamos a modificar el modelo del apartado anterior:
 - Crearemos una función que genere la señal de referencia (preámbulo)
 - Añadiremos canal multicamino
 - En recepción estimaremos la $H(f)$ del canal y la compensaremos

- Señal de entrenamiento:

- Creamos una función:

- Genera valores BPSK conocidos para cargar todas las portadoras, tanto de datos como de pilotos
 - Se obtiene con la iFFT la señal temporal de entrenamiento (`preT`)
 - La señal temporal de salida contiene dos copias de la parte útil del símbolo OFDM generado
 - Este formato es similar al de WLAN (habitualmente se incluye un CP doble)
 - Devuelve también los valores BPSK de referencia para poderlos usar en la estimación del canal (`Xpre_da`, `Xpre_pi`)

```
function [preT,Xpre_da,Xpre_pi] = ref_ofdm(Nfft,Ncp,I_nu,I_pi)
```

```
    rng(35)
```

```
    Npi = length(I_pi);
```

```
    Nda = Nfft - Npi - length(I_nu);
```

```
    Xpre_da = randi([0 1],Nda,1)*2 -1;
```

```
    Xpre_pi = randi([0 1],Npi,1)*2 -1;
```

```
    ref = ofdmmod(Xpre_da,Nfft,Ncp,I_nu,I_pi,Xpre_pi);
```

```
    preT = [ref; ref(end-Nfft+1:end)];
```

- **Canal multicamino y estimación de $H(f)$ (i):**

- Para probar el método de estimación del canal:

1. Generamos una respuesta al impulso del canal
2. Generamos la señal OFDM preámbulo
3. Filtramos la señal por el canal y se añade ruido
4. En el receptor se promedian las dos copias temporales del preámbulo para reducir el ruido
5. Aplicamos el receptor OFDM al preámbulo promediado para obtener los valores BPSK recibidos (datos y piloto por separado)
6. Estimamos la $H(f)$ para cada portadora según: $\hat{H}_k = \frac{Y_k}{A_k} = H_k + W_k$

```
hcanal = [0.2 1 -0.2 0.5 0.1];
```

```
[preT,Xpre_da,Xpre_pi] = ref_ofdm(Nfft,Ncp*2,I_nu,I_pi);  
preT_canal = filter(hcanal,1,preT); % Paso 3 sin ruido
```

```
pini1 = Ncp*2;  
pini2 = pini1+Nfft;  
pre1 = preT_canal(pini1+(1:Nfft));  
pre2 = preT_canal(pini2+(1:Nfft));  
preT_pro = (pre1+pre2)/2;
```

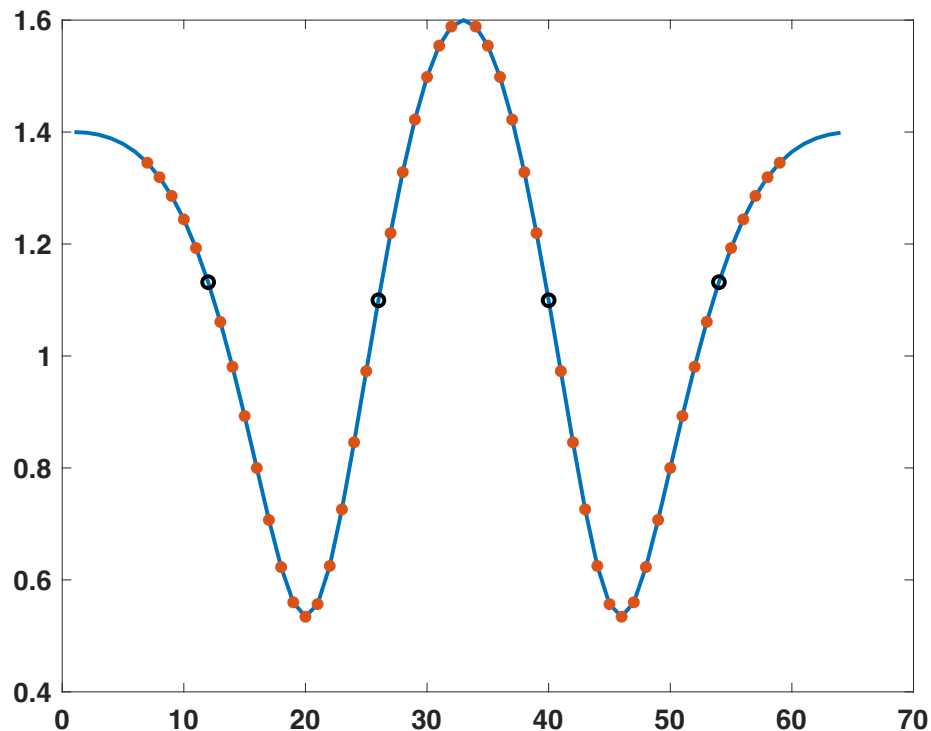
% Paso 4

- **Canal multicamino y estimación de $H(f)$ (ii):**

- Paso 6: estimación del canal:

```
[Ypre_da,Ypre_pi] = ofdmdemod(preT_pro,Nfft,0,0,I_nu,I_pi);  
Hest_da = Ypre_da./ Xpre_da;  
Hest_pi = Ypre_pi./ Xpre_pi;
```

- Podemos comparar las $H(f)$ del canal y estimadas:



```
Hcanal = fft(hcanal,Nfft);  
I_dat=setdiff(1:64,[I_nu(:); I_pi(:)]);  
figure(10)  
plot(1:64,fftshift(abs(Hcanal)),I_dat,...  
abs(Hest_da),'*',I_pi,abs(Hest_pi),'ok')
```

- **Compensación del canal (i):**

- Para recuperar la señal transmitida invertimos el canal estimado y lo multiplicamos por la señal recibida (tras la FFT):

$$\hat{X}_k = \frac{Y_k}{\hat{H}_k}$$

- Una vez estimado el canal, para simular una transmisión de señal OFDM con ecualización:
 1. Generamos la señal OFDM con datos (seguimos sin usar los tonos piloto)
 2. La pasamos por el mismo canal multicamino
 3. Obtenemos los datos recibidos Y_{da}
 4. Creamos una matriz con el ecualizador de N_{Sofdm} columnas repitiendo N_{Sofdm} veces la inversa del canal estimado
 5. Multiplicamos la matriz con los datos recibidos por la matriz de ecualización
 6. Obtenemos los bits a partir de los datos ecualizados
 7. Calculamos el error

- **Compensación del canal (ii):**

```

Mqam = 16;
NSofdm = 10; %Número de símbolos OFDM;
Ncp = 16;

Bits_T = randi([0 1],Nda*log2(Mqam),NSofdm);
Xda = qammod(Bits_T,Mqam,InputType="bit",UnitAveragePower=true);
Xpi = zeros(Npi,NSofdm);

sofdm = ofdmmod(Xda,Nfft,Ncp,I_nu,I_pi,Xpi);
sofdm_canal = filter(hcanal,1,sofdm);

[Yda,Ypi] = ofdmdemod(sofdm_canal,Nfft,Ncp,Ncp,I_nu,I_pi);

eq = 1./Hest_da; % Calcula ecualizador para un símbolo OFDM
EQ = repmat(eq,1,NSofdm); % Genera matriz ecualizador para Nsofdm símbolos
Yda_eq = Yda .* EQ;

Bits_R = qamdemod(Yda_eq,Mqam,OutputType="bit",UnitAveragePower=true);
[n_er,ber] = biterr(Bits_T,Bits_R)

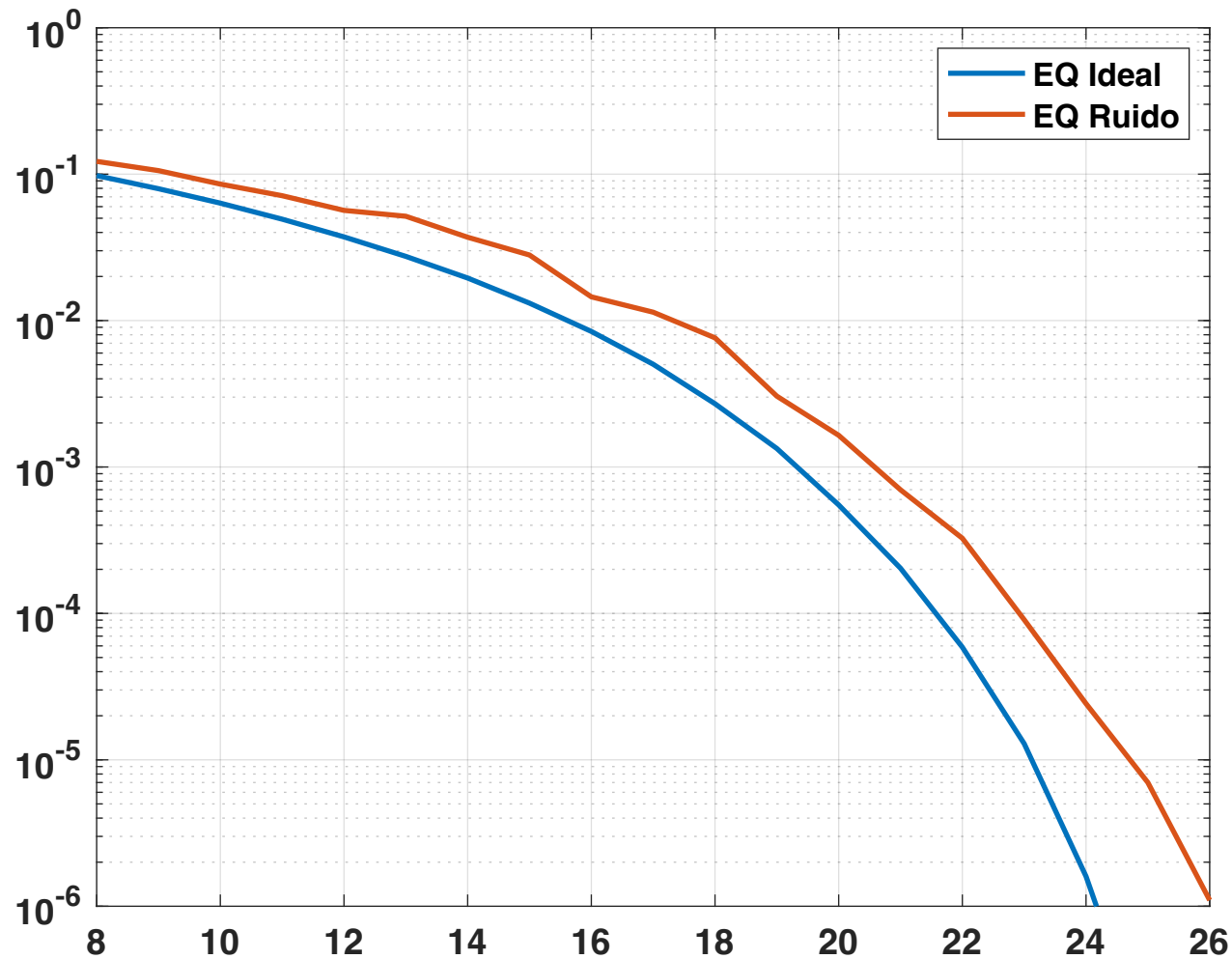
scatterplot(Yda(:)),title('Yda')
scatterplot(Yda_eq(:)),title('Yda\_eq')

Bits_R = qamdemod(Yda,Mqam,OutputType="bit",UnitAveragePower=true);
[n_er,ber] = biterr(Bits_T,Bits_R) % Comprueba errores sin ecualizar

```

Ejercicio:

Repetir la simulación de BER frente a SNR en canal multicamino, comparando las prestaciones de la estimación del canal sin ruido (ideal) frente a hacerla con ruido (condiciones reales).

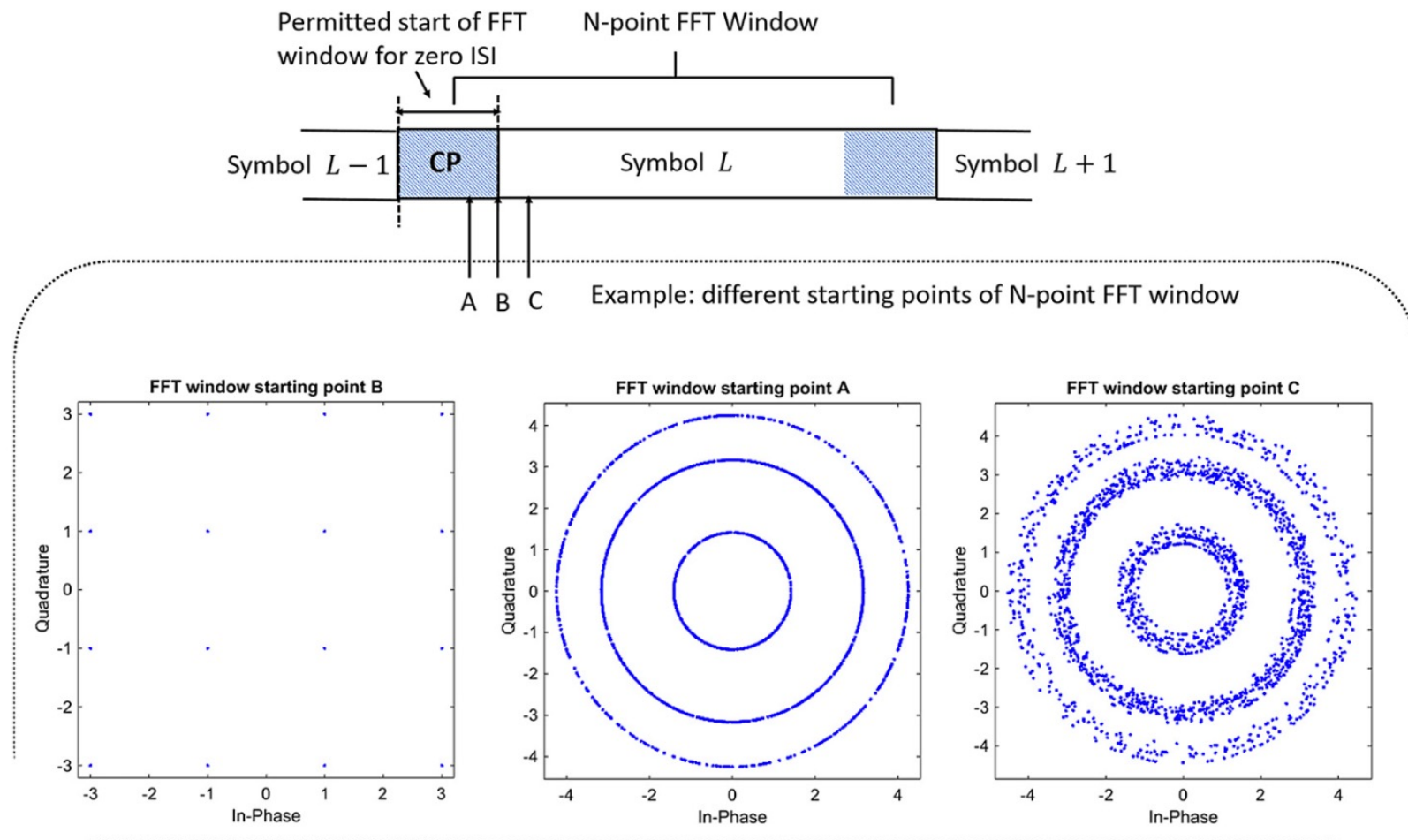


Estructura de la simulación:

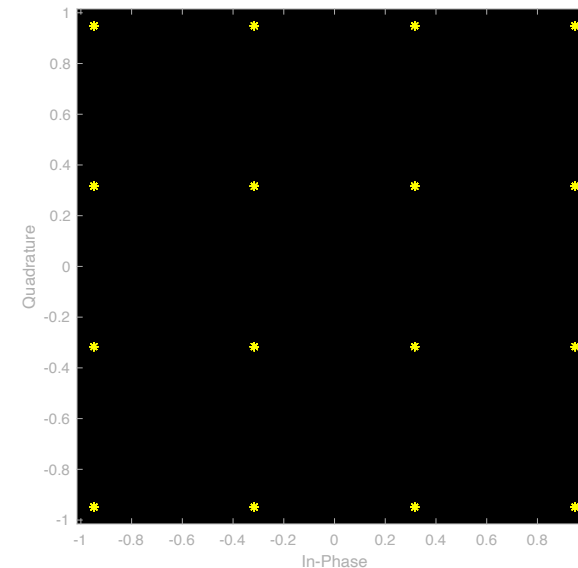
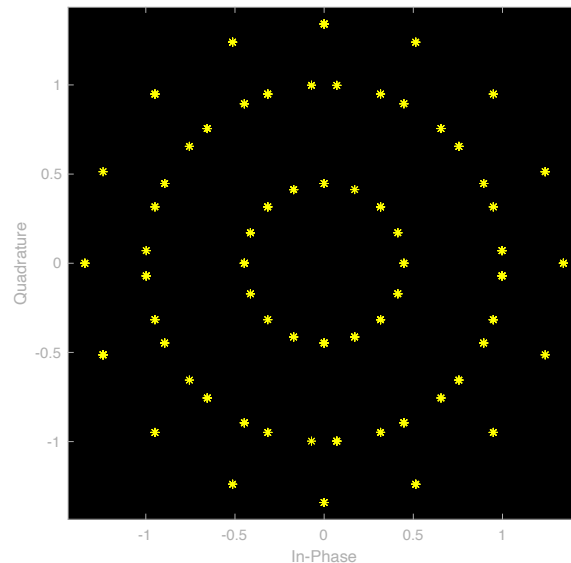
1. Define todos los parámetros para generar OFDM como en WLAN
2. Calcula el NSofdm necesario para transmitir 1e6 bits con 16QAM (4 bits):
 - Cada símbolo OFDM transporta = 4 bits · Num. portadoras datos
3. Genera los bits y úsalos para crear los símbolos OFDM
4. Define SNRdB = 8:26 y un vector BER de ceros del mismo tamaño
5. Crea el preámbulo y define el canal múlticamino
6. Filtra con el canal el preámbulo y la señal OFDM.
7. Estima el ecualizador ideal con el preámbulo sin ruido
8. Crea un bucle (`for k=1:L`) para recorrer los índices de SNRdB:
 - A. Con `awgn` añade ruido a la señal OFDM tras el canal, también al preámbulo
 - B. Estima el ecualizador con el preámbulo ruidoso
 - C. Demodula la señal OFDM que tiene ruido añadido
 - D. Ecualiza los símbolos recibidos con el ecualizador ideal (paso 7) y con el estimado con el preámbulo ruidoso (paso 8.B)
 - E. Obtén los bits recibidos para ambos ecualizadores
 - F. Calcula el BER(k) para ambos ecualizadores
9. Dibuja la gráfica de BER vs. SNR

3. Efectos errores sincronización

- Cambio en el instante inicial de la ventana OFDM:

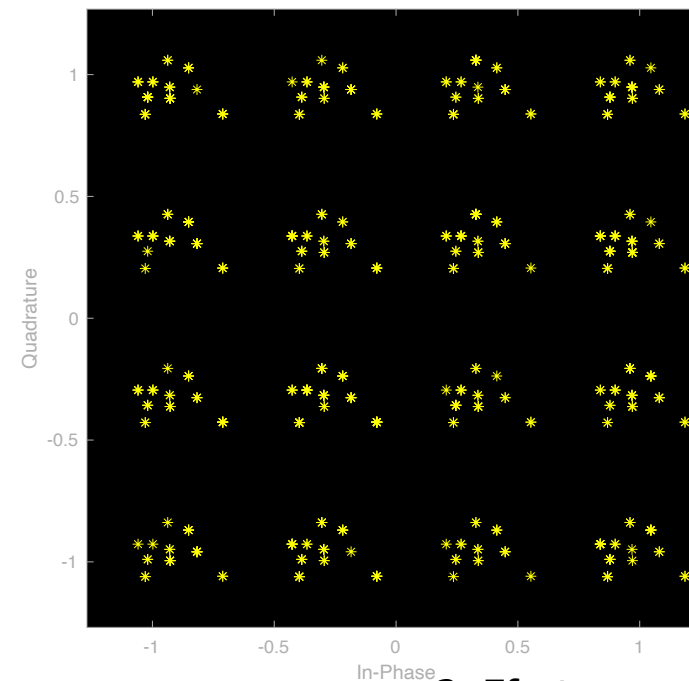
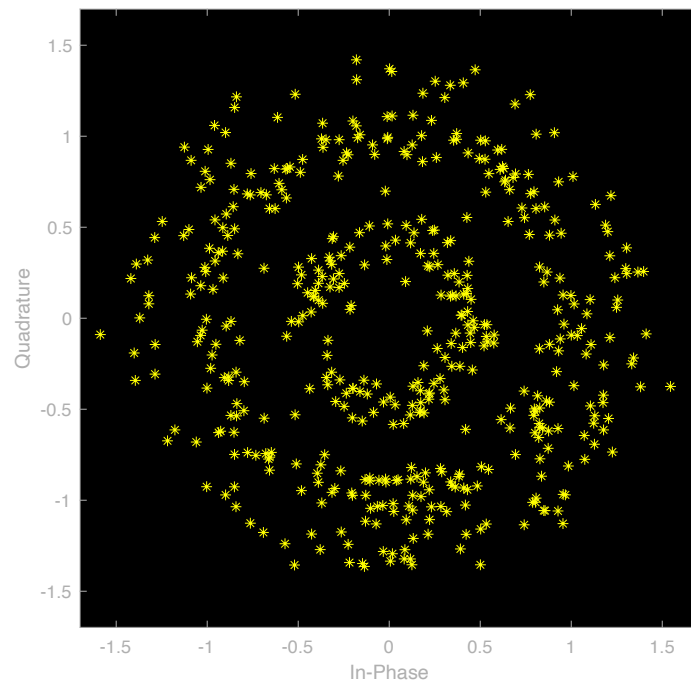


- Avance en el instante inicial de la ventana OFDM:
 - Partimos de los ejemplos de la primera sesión:
 - Carga las variables para configurar WLAN.
 - Genera 10 símbolos OFDM con 16-QAM, y también un preámbulo.
 - Retardamos 4 muestras la señal OFDM y la demodulamos
 - Retardo de un vector **x**: `xret=zeros(4,1); x(1:end-4);`
 - Con `scatterplot(Yda(:))` vemos cómo rota la constelación.



- Compensación en recepción de la rotación :
 - Retarda 4 muestras el preámbulo y estima el canal usando el preámbulo retardado. Genera el ecualizador y úsalo.
 - Dibuja la constelación

- Retraso en el instante inicial de la ventana OFDM:
 - Genera 10 símbolos OFDM y un preámbulo
 - Elimina la primera muestra de señal y preámbulo y añade un cero al final para que los nuevos vectores tengan el mismo tamaño
 - Equivale a un retraso en el inicio de la ventana FFT del receptor de 1 muestra
 - Demodula la señal y comprueba que la constelación gira e incluye ICI.
 - Usa el preámbulo modificado para estimar el canal, genera el ecualizador y úsalo para compensar la rotación.
 - Comprueba que se compensa la rotación, pero no la dispersión causada por la ICI.



3. Efectos errores sincronización

- Offset de frecuencia (i):

- Genera 1 símbolo OFDM y lo modulas con una $e^{j2\pi f_0 n}$
 - Con $f_0=1/64*0.05$
 - Se observa la dispersión por la ICI y el giro (CPE)

```
NSofdm = 1;
Mqam = 4;
```

```
BitsPi = randi([0 1],Npi,NSofdm);
Xpi = pskmod(BitsPi,2);
```

```
Bits_T = randi([0 1],Nda*log2(Mqam),NSofdm);
Xda = qammod(Bits_T,Mqam,InputType="bit",UnitAveragePower=true);
scatterplot(Xda(:)),title('Xda')
```

```
sofdm = ofdmmod(Xda,Nfft,Ncp,I_nu,I_pi,Xpi);
```

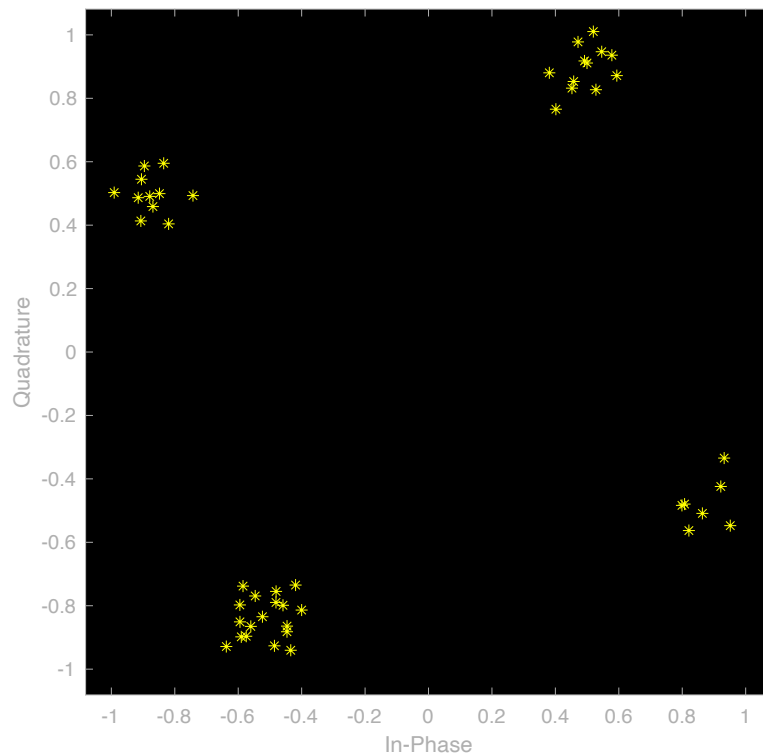
```
CF0 = 1/64*0.05;
Ls=length(sofdm);
```

```
sofdm_cfo = sofdm.* exp(+1j*2*pi*CF0*(1:Ls)');
[Yda,Ypi] = ofdmdemod(sofdm_cfo,Nfft,Ncp,Ncp,I_nu,I_pi);
scatterplot(Yda(:))
```

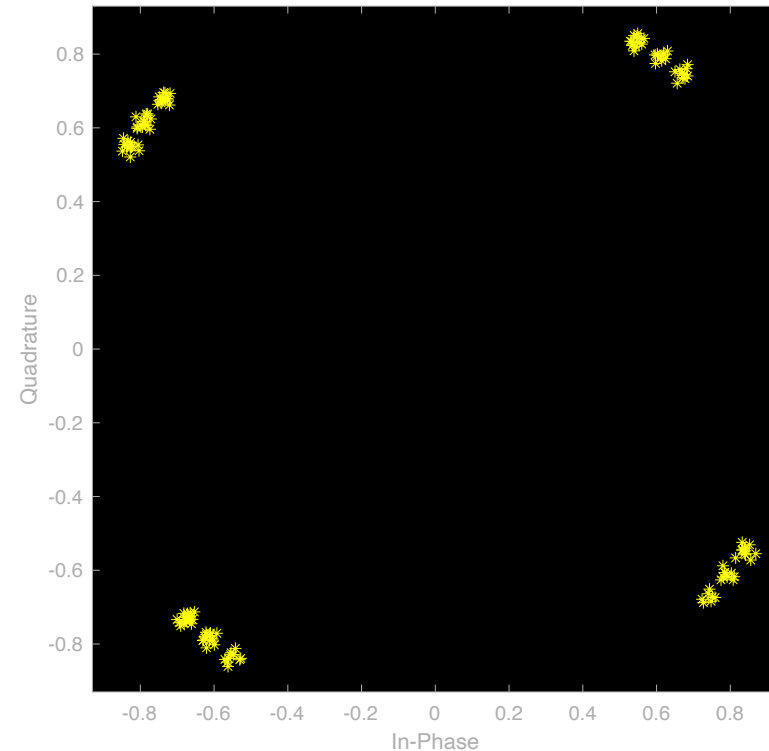
- Offset de frecuencia (ii):

- Repite para 3 símbolos OFDM y con una $f_o = 1/64 * 0.01$
 - Con un CFO menor se reduce la ICI
 - Se ve que para cada símbolo OFDM hay un incremento en el CPE

1 símbolo, CFO = 0.05/64



3 símbolos, CFO = 0.01/64



4. Métodos de sincronización

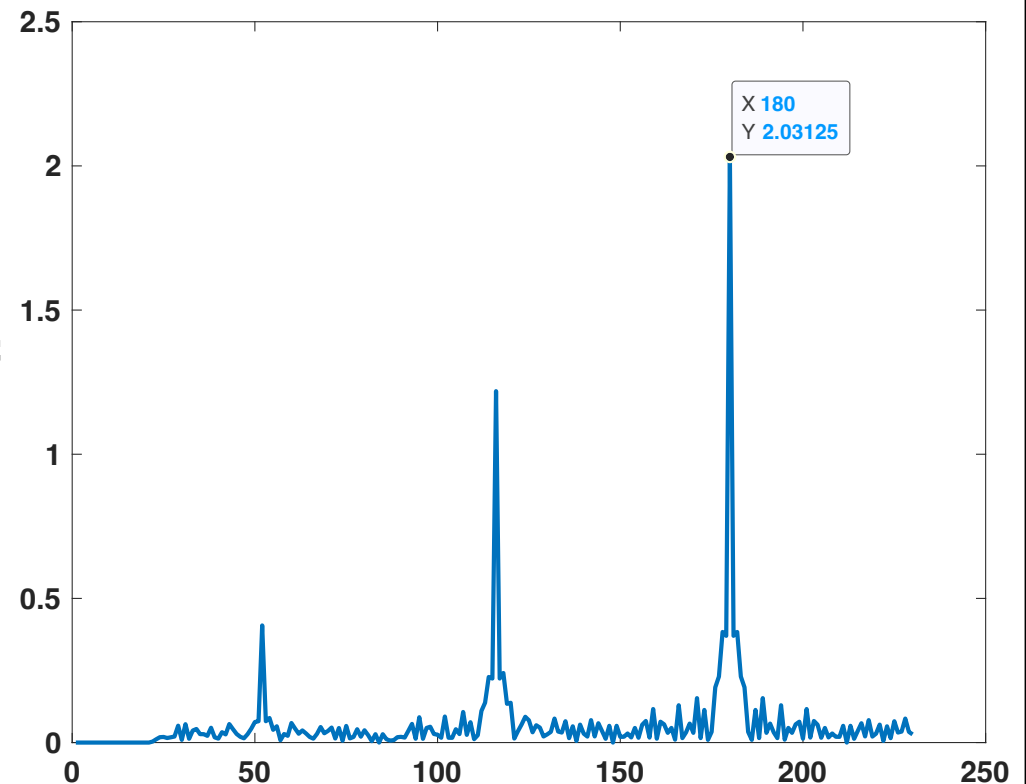
- Transmisión a ráfagas vs. continua
 - Ráfagas (WLAN) requiere detección de paquete:
 - Estima energía en dos ventanas consecutivas y las compara, si hay un cambio brusco indica presencia del inicio del paquete.
 - Continua.
 - Broadcast (DVB, DAB): búsqueda inicio símbolo OFDM usando patrón de repetición por el uso de CP
 - LTE o 5G: búsqueda símbolo OFDM de sincronización
- Sincronización de tiempo basada en preámbulo:
 - En WLAN tiene estructura repetitiva: uso autocorrelación
 - El método más empleado es el propuesto por Schmidl y Cox.
 - Para CFO bajos se usa la correlación cruzada:
 - En LTE y 5G es la solución para detectar los símbolos de sincronización
 - En WLAN tras corregir CFO se refina con correlación

- Sincronización con preámbulo (símbolo) conocido:
 - Si $p(t)$ es el preámbulo transmitido, la correlación es equivalente a filtrar por $p^*(-t)$.
 - Da un pico en la última muestra del preámbulo.
 - Por ejemplo, para los parámetros WLAN:

```
% Preambulo
preT = ref_ofdm(Nfft,Ncp*2,I_nu,I_pi);

% Filtro correlador
filCOR = conj(preT(end:-1:1));

trama = [zeros(20,1); preT; zeros(50,1)];
salco = filter(filCOR,1, trama);
figure(10),plot(abs(salco))
```



- Estimación del offset de frecuencia (i):
 - Sea s_n la señal transmitida, r_n la recibida con CFO:

$$r_n = s_n e^{j2\pi f_o n}$$

- Planteamos la siguiente operación en recepción:

$$Z = \sum_{n=0}^{L-1} r_n \cdot r_{n+D}^*$$

- Queda en función del CFO: $Z = e^{-j2\pi f_o D} \sum_{n=0}^{L-1} |s_n|^2$
- Por lo que puede estimarse como:

$$\hat{f}_o = -\frac{1}{2\pi D} \arg\{Z\} \quad \text{con } \arg\{Z\} \text{ entre } [-\pi, \pi)$$

- Estimación del offset de frecuencia (ii):
 - Este método estima correctamente $\text{CFO} < 1/2D$
 - Si usamos las dos copias del preámbulo, como están separadas por 64 muestras, el estimador tiene un rango $0.5 \cdot 1/64$, es decir, media separación entre portadoras.
 - Utiliza este método para estimar un CFO de $0.2/64$ en el preámbulo:

$$\text{CFO} = 1/64 * 0.2$$

$$\text{preT_cfo} = \text{preT} .* \exp(+1j * 2 * \pi * \text{CFO} * (1:160)');$$

- Para conjugar usa `conj`, y para obtener el ángulo usa `angle`.
- r_n es `pre1`, y r_{n+D} es `pre2`
- Normalmente en condiciones reales la estimación no es exacta: al corregirla queda un CFO residual
 - El CFO residual provoca que en cada símbolo OFDM todas las portadoras roten el CPE, es una rotación acumulativa: con el tiempo los símbolos se salen de su cuadrante y se producen errores en la detección.
 - Es necesario hacer un `seguimiento` del offset residual.

- Seguimiento del offset de frecuencia (i):
 - En cada símbolo OFDM se utilizan las portadoras piloto para estimar el CPE de ese símbolo.
 - Para el símbolo m con los pilotos recibidos R_k teniendo en cuenta los pilotos transmitidos P_k y la respuesta del canal H_k :

$$R_k = H_k P_k e^{j2\pi f_\Delta n}$$

- Se puede estimar el CPE residual:

$$\hat{\omega}_\Delta = \arg \left\{ \sum_{n=0}^{L-1} R_k \cdot (\hat{H}_k P_k)^* \right\}$$

- Para canal perfectamente estimado la ecuación anterior da:

$$\hat{\omega}_\Delta = \arg \left\{ e^{j2\pi f_\Delta n} \sum_{n=0}^{L-1} |H_k|^2 \right\}$$

- Seguimiento del offset de frecuencia (ii):
 - Usa este estimador con una señal de 3 símbolos OFDM para estimar y corregir el caso de un CFO= 0.01/64 en canal plano
 - Usamos el ejercicio final del apartado 3:

```

Hest_pi = ones(Npi,1);    % en este ejemplo el canal es plano

[Yda,Ypi] = ofdmdemod(sofdm_cfo,Nfft,Ncp,Ncp,I_nu,I_pi);
Yda_eq = Yda;    % en este ejemplo no hace falta ecualizar: canal plano

HkPk = Xpi .* repmat(Hest_pi,1,NSofdm);

CFO_seg = angle(sum(Ypi.*conj(HkPk))); %Tamaño: 1 x NSofdm

exp_CFO = repmat(exp(-1j*CFO_seg),Nda,1); %Tamaño: Nda x NSofdm

Yda_eq_cfo = Yda_eq.*exp_CFO; %todas portadoras del símbolo tienen mismo CFO

scatterplot(Yda_eq_cfo(:))

```

5. Receptor OFDM sincronizado

- Baja de PoliformaT los archivos: `test_simu_ofdm_trx.m` y `Bits_transmission_OFDM.mat`
 - Genera una trama de señal OFDM usando bits de una imagen
 - Y añade canal multicamino, ruido y CFO configurando las variables iniciales.
- Baja también: `guia_rcx_ofdm.m`
 - Contiene la estructura para implementar un receptor sincronizado.
 - Debes añadir las etapas que faltan para completar el receptor
- Comprobación de funcionamiento:
 - Ejecuta transmisor y receptor con $\text{SNR}=100$; $\text{hcanal}=1$; $\text{CFO}=0$
 - Si funciona, añade ruido y $\text{CFO}=0.2/64$; si funciona, luego añade también canal multicamino.

- Recepción con señal capturada por RTL-SDR
 - Carga la señal `Captura_RTL_ofdm.mat`
 - Haz un `plot` del valor absoluto de la señal `sig_rcx` y compárala con la generada por `test_simu_ofdm_trx.m`
 - Prueba a recibir la trama capturada con el RTL-SDR usando tu receptor OFDM
 - Si el receptor obtiene una constelación clara y no da errores, o son muy pocos:
 - Calcula los LLR de los símbolos recibidos y ecualizados:

```
llr_rcx = qamdemod(Yda_eq_cfo,Mqam,OutputType="llr",UnitAveragePower=true);
```

- Utiliza la función `reconstruye_jpg.m` para reconstruir la imagen recibida a partir de los LLR.
- Requiere que bajes también el fichero `matriz_LDPC_648_R12.mat` para configurar el decodificador LDPC usado en el transmisor.
- En el directorio de trabajo aparece un fichero `jpg` con la imagen recibida.